

Autumn - 19

1(a)

An algorithm is an ordered set of unambiguous executable steps, defining a terminating process.

Two real life examples where algorithm is applied and efficiency of that algorithm is important —

A teacher sorting their students' papers according to the alphabetical order of their first names. This type of task is similar to the function of a sorting algorithm, like bucket sort. By looking at only the first letter of the first name, we can remove a lot of unnecessary information. This is an automated process that makes sorting more efficient.

Finding a library book is another real life example of using an algorithm. There are multiple

Processes for successfully finding a library book, whether it's by utilizing the library's computer catalog system, identifying genre labels on shelves and finding the book by Subject or author etc. Regardless of the process used, if it can be defined and replicated by others then it is an algorithm.

~~Ans. is~~ 1(b)

```

for (int i = 1; i <= n; i++)
    printf("Algorithm");

```

Diagram illustrating the loop structure with annotations:

- ① points to the initialization `i = 1`.
- ② points to the condition `i <= n`.
- ③ points to the increment `i++`.
- ④ points to the loop body `printf("Algorithm");`.

① $\longrightarrow$ 1 time execution

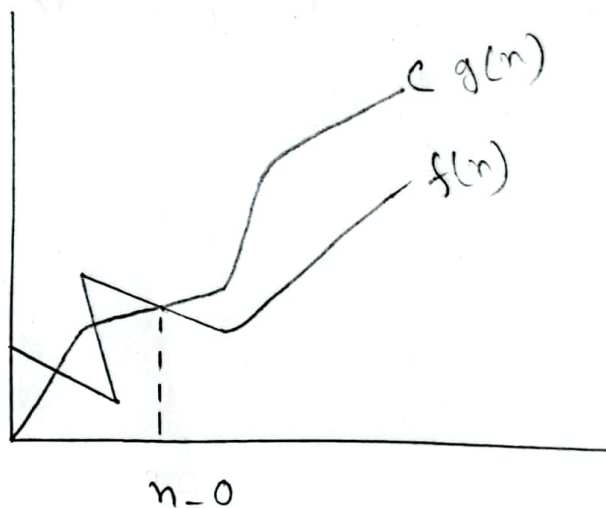
②, ③, ④ $\rightarrow n$ times

Asymptotic Notation

$$T(n) = 3n + 1 \quad \underbrace{\hspace{2cm}}_{f(n)}$$

$$T(n) = O(n) \quad \underbrace{\hspace{2cm}}_{g(n)}$$

Big-O notation



$$f(n) = O(g(n))$$

Here, $f(n) = 3n + 1$; $g(n) = n$

$$\therefore f(n) \leq c \cdot g(n)$$

$$\Rightarrow 3n + 1 \leq 4n$$

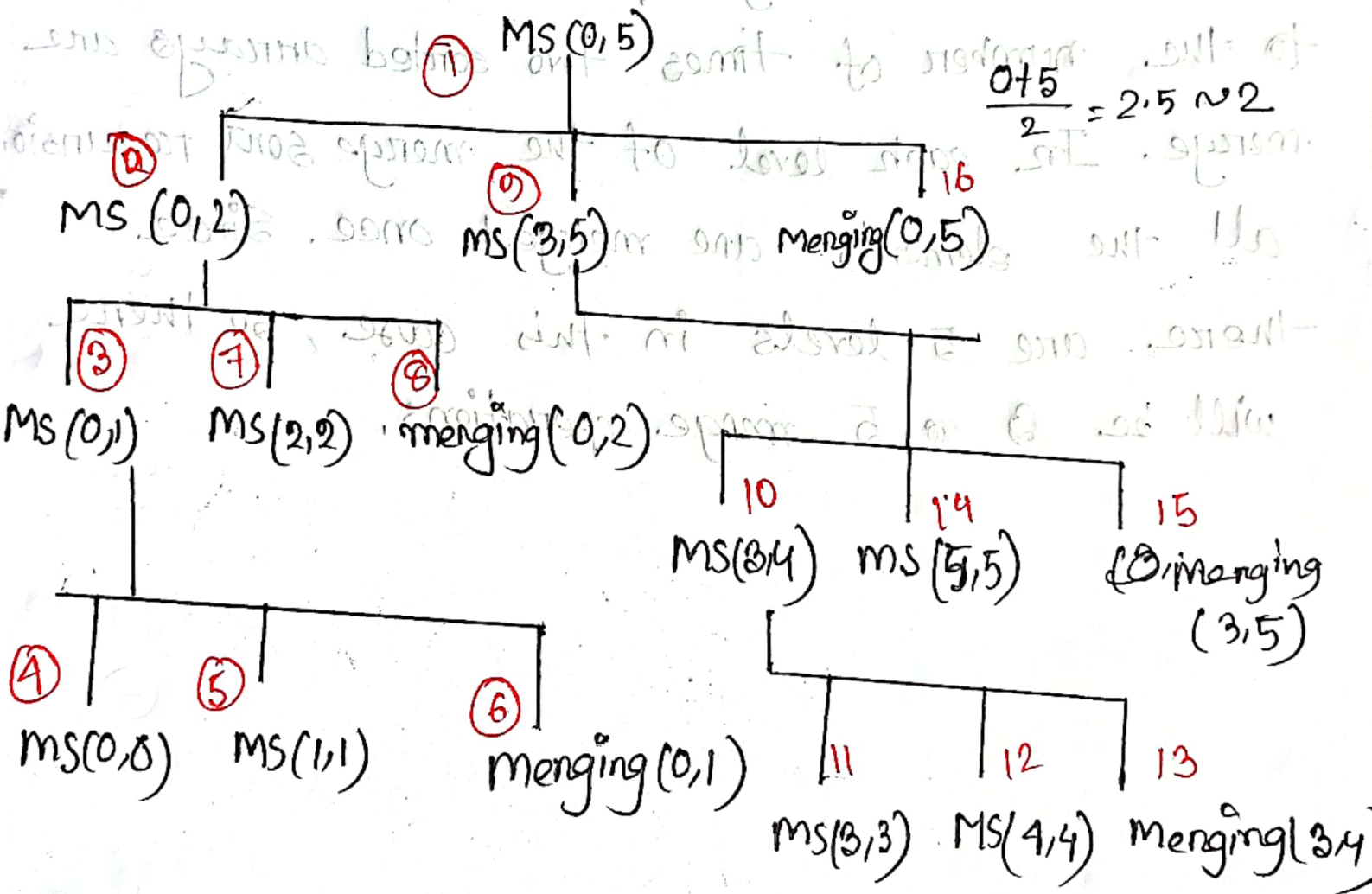
If $n = 1$, then $4 \leq 4$; True and it is true for all values $n \geq 1$. That is, $f(n) = O(n)$ for $c = 4$ and $n \geq 1$.

Ans to the ques no : 1(c)

An array for merge sort

$A = \langle 9, 7, 8, 3, 2, 1 \rangle$

0	1	2	3	4	5
9	7	8	3	2	1



2(a)

Proof:-

From the preceding discussion, it suffices to determine the height of a decision tree in which each permutation appears as a reachable leaf. Consider a decision tree of height h with l reachable leaves corresponding to a comparison sort on n elements. Because each of the $n!$ permutations of the input appears as one or more leaves, we have $n! \leq l$. Since a binary tree of height h has no more than 2^h leaves, we have

$$n! \leq l \leq 2^h,$$

which by taking logarithms, implies

$$h \geq \log(n!)$$

$$h = \Omega(n \log n)$$

2(b)

Here,

$A = \{9, 2, 5, 4, 3, 1, 4, 6\}$

and last element = pivot = 6

let, start = 1st element and end = last element

~~7, 6, 1~~ 9, 2, 5, 4, 3, 1, 4, 6

start

last element
and
pivot

Now we have to apply two conditions:

1. $start \leq pivot$ ~~start~~ then $start++$

2. $end > pivot$ then $end++$

0	1	2	3	4	5	6	7
9	2	5	4	3	1	4	6
start							end

$9 < 6$
 $6 > 6$

0	1	2	3	4	5	6	7
6	2	5	4	3	1	4	9
					end	start	

(swapping)

$6 < 6$

$2 < 6$

$5 < 6$

$4 < 6$

$9 < 6^x$

0	1	2	3	4	5	6	7
4	2	5	4	3	1	6	9
						pivot	

This is the 1st partition of this given array.

$9 > 6$
 $4 > 6$

Answer to the Question : 02 (c)

After analyzing the time complexities of insertion, merge and quick sort, sorting an array containing 10^5 numbers in sorted order;

Insertion : This is a well-suited algorithm for an array already sorted in ascending order. It performs efficient comparisons and shifts, requiring approximately $O(n)$ operations for an array of size 10^5 . Therefore, insertion sort would take minimum time.

Merge : Merge sort's time complexity is consistently $O(n \log n)$ regardless of initial order. It divides the array into equal halves, sorts them, and then merges them back together. For an array of size 10^5 , merge sort would consistently take a moderate amount of time.

Quick : Quick sort algorithms' efficiency depends on pivot selection. In the best case, it achieves an average time complexity of $O(n \log n)$. However, for an array already sorted, poor pivot choices could lead to worst case behaviour of $O(n^2)$, making quick sort taking maximum time.

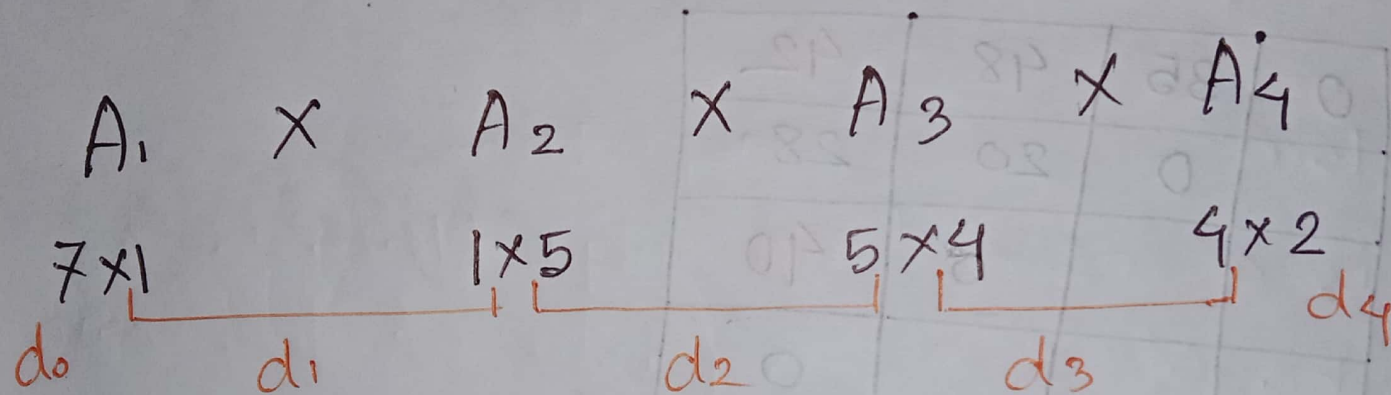
For an array of 10^5 numbers in sorted order, insertion sort will be fastest, due to its efficient handling of nearly sorted data, and Quicksort might take the longest time.

= X =

3(a)

The given sequence, $\{7, 1, 5, 4, 2\}$

$$\text{so, } d_0 = 7, d_1 = 1, d_2 = 5, d_3 = 4, d_4 = 2$$



from m table,

$$A[1,1] = 0, A[2,2] = 0, A[3,3] = 0, A[4,4] = 0$$

$$A[1,2] = 35, A[1,3] = 48, A[2,3] = 20, A[2,4] = 28$$

$$A[1,4] = ?$$

$$A[3,4] = ?$$

$$A[3,4] = \min_{i \leq k < j} \left\{ \begin{aligned} &k=3; m[3,3] + m[4,4] + d_2 \times d_3 \times d_4 \\ &= 0 + 0 + 5 \times 4 \times 2 \\ &= 40 \end{aligned} \right\}$$

$$A[1,4] = ?$$

Now,

$$A[i,j] = \min_{i \leq k \leq j}$$

$$\left\{ \begin{array}{l} k=1, m[1,1] + m[2,4] + d_0 \times d_1 \times d_4 \\ 0 + 28 + 7 \times 1 \times 2 = 42 \end{array} \right.$$

$$\left\{ \begin{array}{l} k=2, m[1,2] + m[3,4] + d_0 \times d_2 \times d_4 \\ 35 + 40 + 7 \times 5 \times 2 = 145 \end{array} \right.$$

$$\left\{ \begin{array}{l} k=3, m[1,3] + m[4,4] + d_0 \times d_3 \times d_4 \\ 48 + 0 + 7 \times 4 \times 2 = 10 \end{array} \right.$$

So, the m table and s table is

	1	2	3	4
1	0	35	48	42
2		0	20	28
3			0	40
4				0

m table

	1	2	3	4
1	0	1	1	1
2		0	2	3
3			0	3
4				0

s table

parentheses:

$$(A_1)((A_2 A_3) A_4)$$

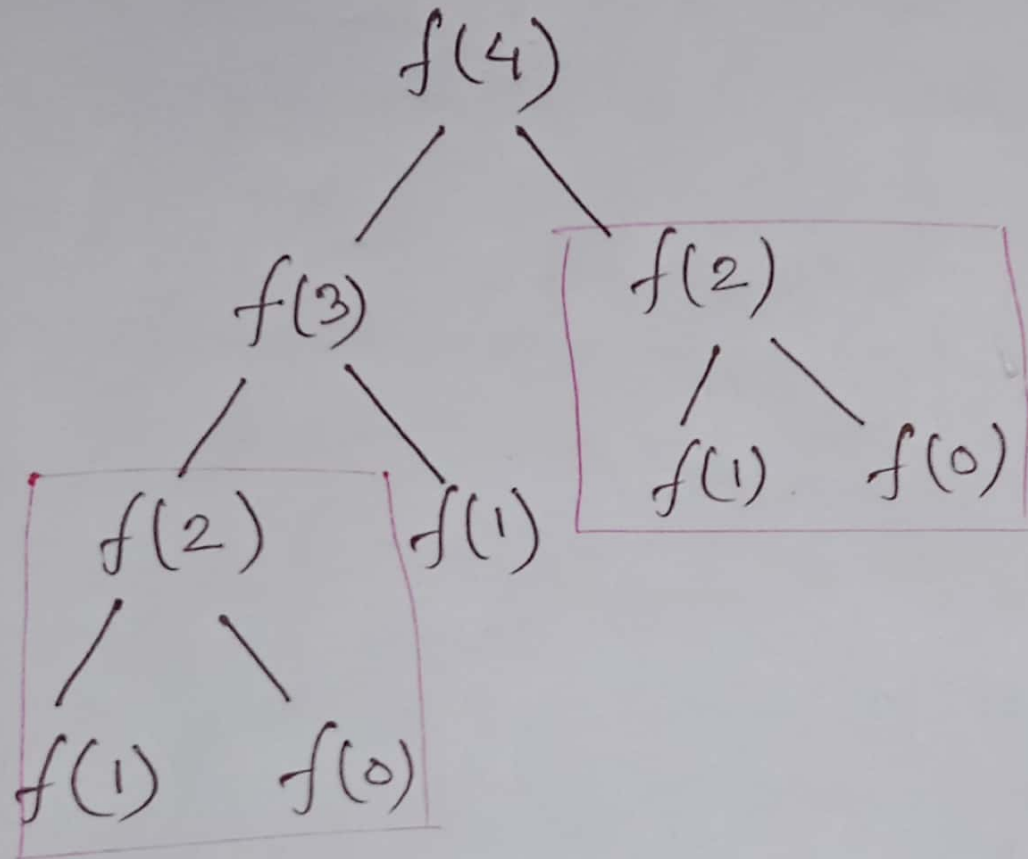
3(b)

Diff. between problem and subproblem

problem	subproblem
1. Problem is a one kind of task where we have to generate a solution.	1. A subproblem is a subparts of the main problem
2. problem is goal directed	2. Subproblem is main problem directed.
3. problem has one solution	3. Subproblems add up to the total problem.

$$f(n) = f(n-1) + f(n-2)$$

Example:



Answer to the Question: 03 (a).

The Matrix Chain Multiplication problem displays the overlapping subproblem property, a pivotal characteristic in dynamic programming. In this problem, given a sequence of matrix $[A_1, A_2, \dots, A_n]$, the objective is to identify the optimal parenthesization to minimize scalar multiplications during matrix multiplication.

The overlapping subproblem property surfaces due to the recursive nature of the problem. To ascertain the optimal parenthesization for a subsequence $[A_i, A_{i+1}, \dots, A_j]$, various split points k ($i \leq k < j$) are explored. Each split entails solving the same subproblems, including determining the optimal parenthesization of $[A_i, \dots, A_k]$ and $[A_{k+1}, \dots, A_j]$.

For instance, in the sequence $[A_1, A_2, A_3, A_4]$, the subproblem $[A_1, A_2, A_3, A_4]$ is divided into overlapping subproblems:

1) optimal parenthesization of $[A_1]$ and $[A_2, A_3, A_4]$

2) optimal parenthesization of $[A_1, A_2]$ and $[A_3, A_4]$

3) optimal parenthesization of $[A_1, A_2, A_3]$ and $[A_4]$

These subproblems recur with varying parameters, leading to repetitive ~~calculation~~ calculations. Dynamic programming efficiently addresses this by storing

solutions in a table, allowing each subproblem to be computed only once and reused when necessary.

In summary, the Matrix Chain Multiplication problem's overlapping subproblem property emerges from its recursive structure, aligning seamlessly with dynamic programming principles and enabling efficient solutions by sidestepping redundant computations through stored solutions.

==X==

Answer to the Question no :- 4(a)

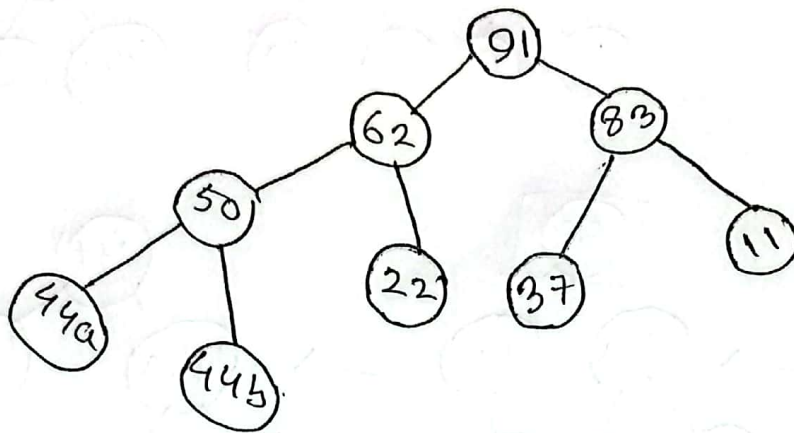
i) It is a complete binary tree. We can represent a binary tree as an array by following some rules. Such that —

- ⊙ The root is at index zero in the array.
- ⊙ The left child of the i^{th} node is at $(2 \times i + 1)^{\text{th}}$ index.
- ⊙ The right child of the i^{th} node is at $(2 \times i + 2)^{\text{th}}$ index.
- ⊙ The parent of the i^{th} node is at $\{(i-1)/2\}^{\text{th}}$ index.

The array representation of the max-heap above is as follows :-

data	91	62	83	50	22	37	11	44
Index	0	1	2	3	4	5	6	7

ii) If we increase the last value that is 44 then the tree will be :-



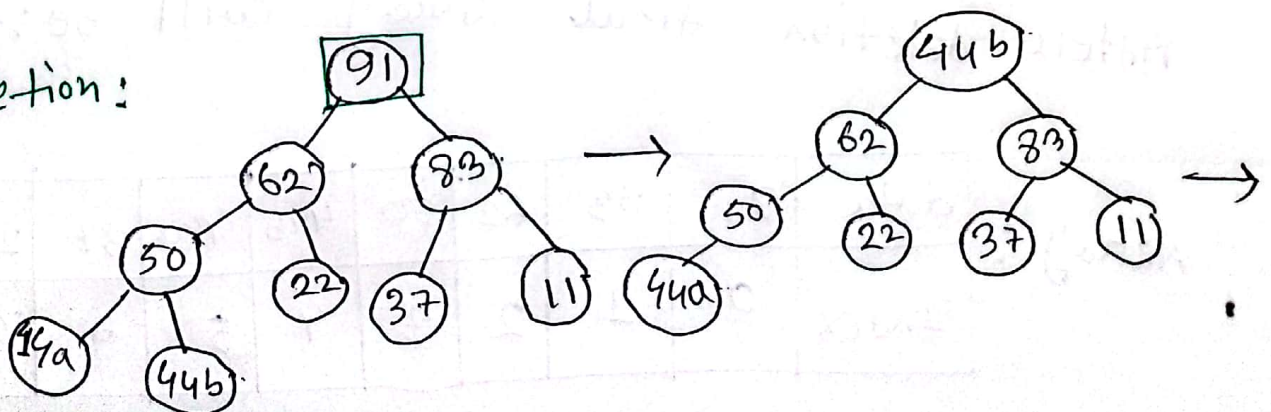
Array :

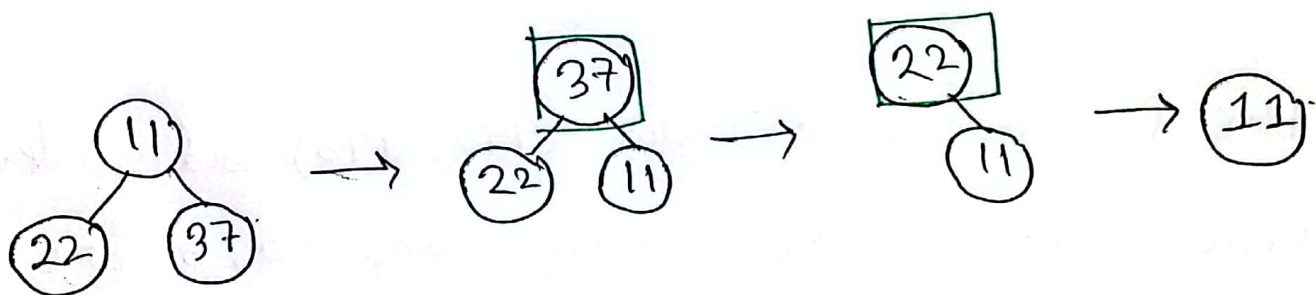
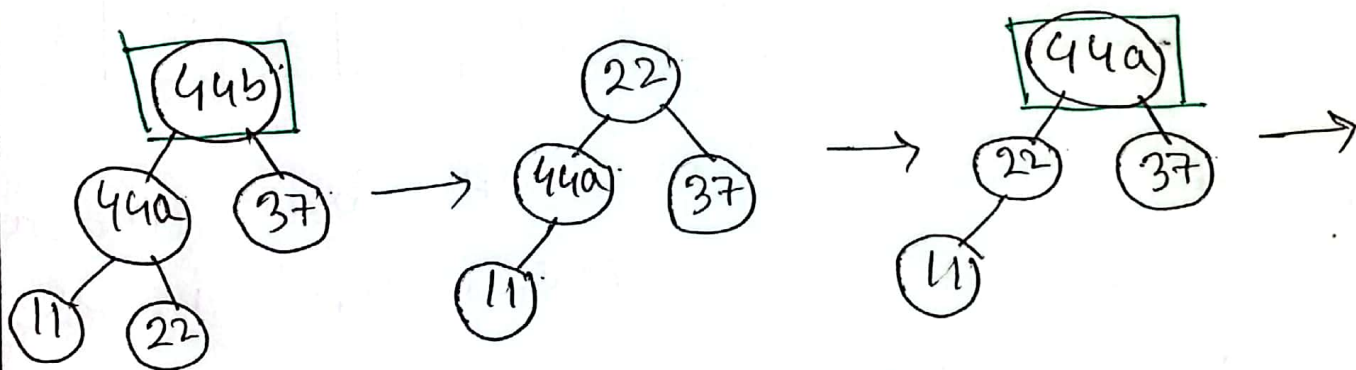
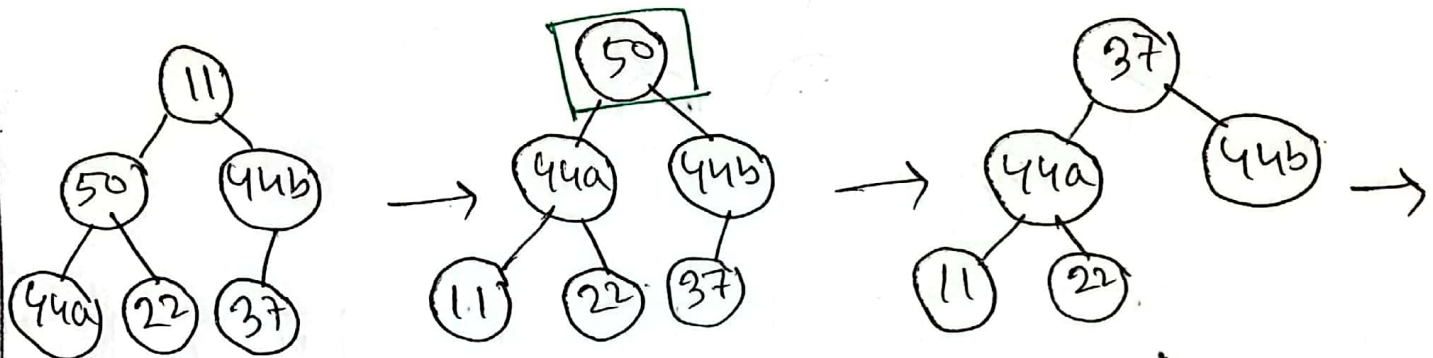
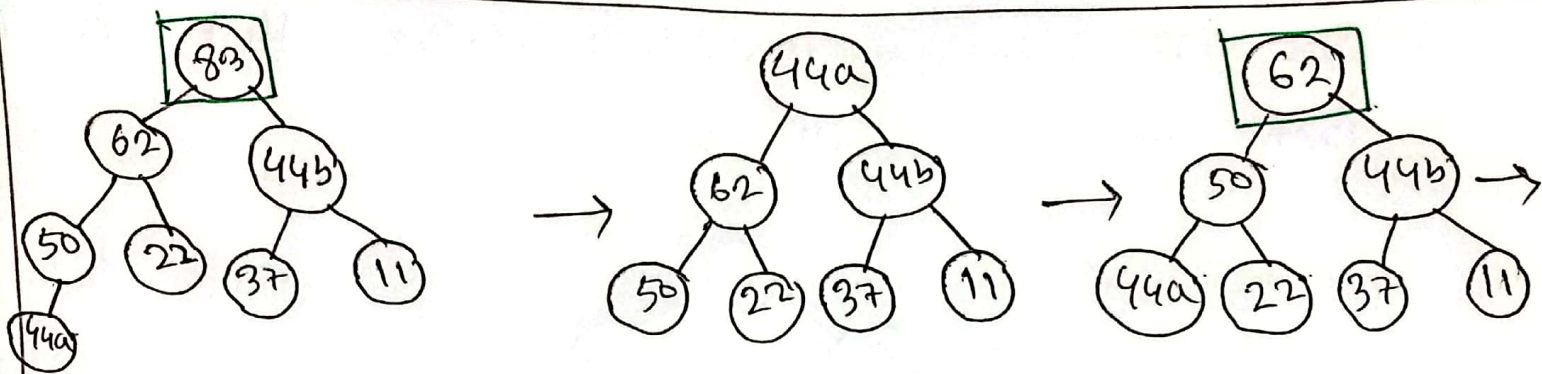
data	91	62	83	50	22	37	11	44a	44b
Index	0	1	2	3	4	5	6	7	8

A sort is unstable when it doesn't preserve the order of same valued input in the resultant output.

Here Unstable sort : Input (44a, 44b) but output (44b, 44a)
 here 44 is the value and a, b are used to denote the order of value 44.

Deletion :





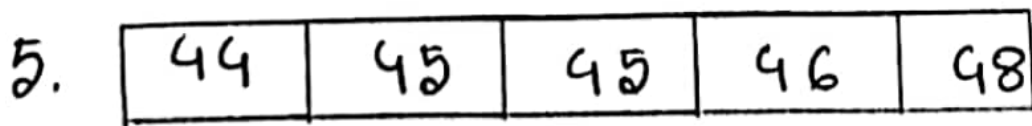
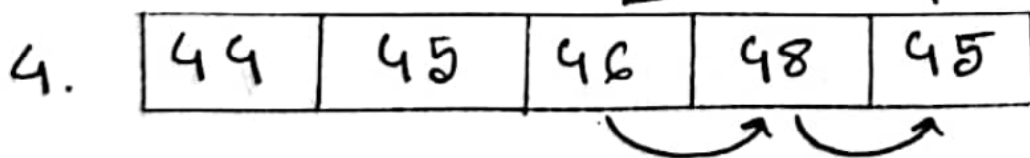
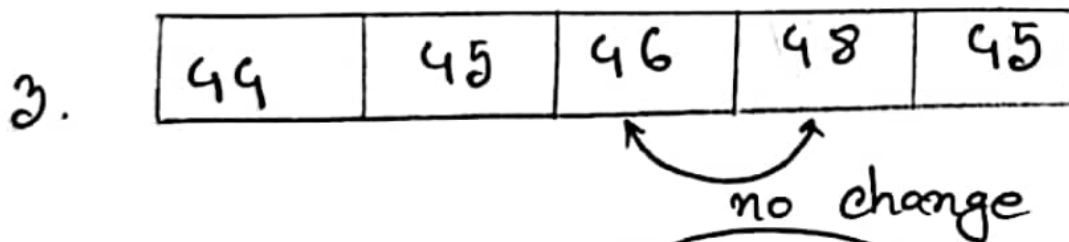
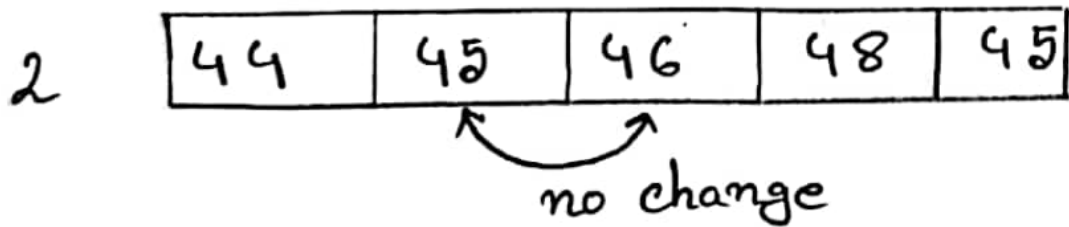
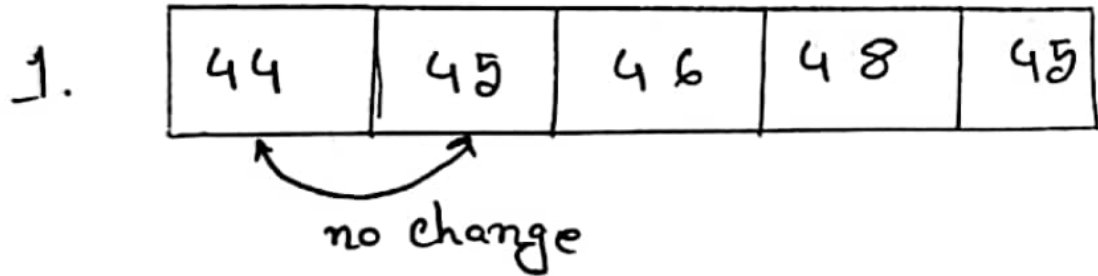
After deletion final array will be:-

Array :

data	91	83	62	50	44b	44a	37	22	11
Index	0	1	2	3	4	5	6	7	8

4(b)

$A = \{44, 45, 46, 48, 45\}$



So, the inner loop of insertion sort will required 5 comparison and one swap

Autumn-19

Ans to the Q.No-4C

In the quicksort algorithm, the recurrence equation represents the time complexity of the algorithm in terms of its sub-problems. When the partitioning step always produces a 5-to-5 proportional split, it means that the pivot is chosen such that roughly half of the elements are smaller and half are larger.

Let's denote the time complexity of quicksort on array of size 'n' as $T(n)$. In this case, the recurrence equation for quicksort can be written as follows:

$$T(n) = 2 \times T(n/2) + O(n)$$

Here's how the equation breaks down -

- The algorithm divides the array into two roughly equal subarrays (5 to 5 split) in each iteration, resulting in two subproblems of size $n/2$.
- The partitioning step takes linear time $O(n)$ because every element needs to be compared to the pivot.

Now, let's solve this recurrence using the Master Theorem, which is a common method for solving recurrence equations of this type.

The Master Theorem has the form:

$$T(n) = a \times T(n/b) + f(n)$$

In our case: $a = 2$ number of subproblems

$b = 2$ size of subproblems

$f(n) = O(n)$ time complexity of partitioning step

The Master Theorem has 3 cases (assuming $a \geq 1, b > 1$)

1. If $f(n) = O(n^c)$ where $c < \log_b(a)$, $T(n) = O(n^{\log_b(a)})$

2. If $f(n) = \Theta(n^{\log_b(a)})$, $T(n) = \Theta(n^{\log_b(a)} \times \log n)$

3. If $f(n) = \Omega(n^c)$, $c > \log_b(a)$, if $a \times f(n/b) = k \times f(n)$ for some constant $k < 1$ and sufficiently large n ,
 $T(n) = \Theta(f(n))$

In our case, $a = 2$, $b = 2$ since $f(n) = O(n)$ and $\log_b(a) = \log_2(2) = 1$, we are in case 1.

Therefore, .
the solution to the recurrence is:

$$T(n) = \Theta(n^1) = \Theta(n)$$

So, the quicksort algorithm with a 5 to 5 proportional split always results in an average case time complexity of $\Theta(n)$ when expressed in asymptotic notation. This indicates linear time complexity, making it quite efficient compared to the general quicksort algorithm with other partitioning strategies.